1.1操作系统的基本概念

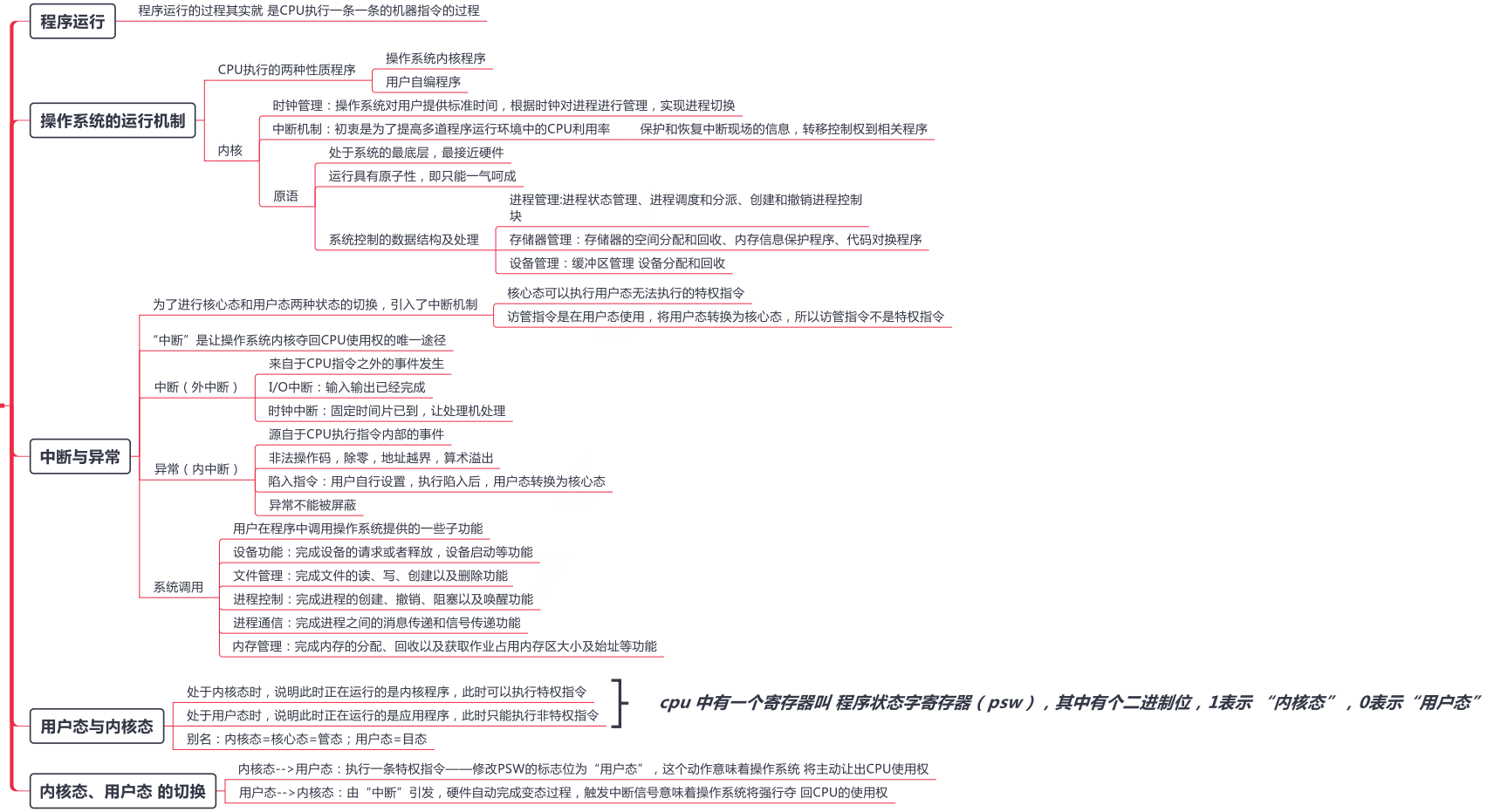


王道论坛
专注于计算机考研的点点滴滴

1.2操作系统的发展与分类



1.3操作系统的运行环境



1.4大内核与微内核

- 大内核
  - 将操作系统的主要功能模块进行集中，从而用以提供高性能的系统服务
  - 优点：各个管理模块之间共享信息，能够有效利用相互之间的有效特性，所有有着巨大的性能优势
  - 缺点：层次交互关系复杂，层次接口难以定义，层次之间界限模糊
- 微内核
  - 背景：随着计算机体系结构的不断发展，操作系统提供的服务越来越多,接口形式越来越复杂
  - 将内核中最基本的功能（如：进程管理）保留在内核，将不需要在核心态执行的功能转移到用户态执行，降低内核设计的复杂性
  - 优点
    - 有效的分离内核与服务、服务与服务、使得他们之间的接口更加的清晰，维护的代价大大降低
    - 各部分可以独立的优化和演进
  - 缺点
    - 性能问题，需要频繁的在核心态和用户态之间进行切换

2.1进程与线程（上）

进程的概念和特征

进程的概念

为了更好地描述和控制程序的并发执行，实现操作系统的并发性和共享性

进程控制块（PCB）：为了更好的描述进程的基本情况和运行状态，进而控制和管理进程

PCB是进程存在的唯一标志

进程的一些典型定义
- 进程是程序一次执行过程
- 进程是一次程序及其数据在处理机上顺序执行时所发生的活动
- 进程是具有独立功能的程序在一个数据集合上运行的过程，是资源分配和调度的独立单位（没有引入线程）

进程的特征

动态性：动态性是进程最基本特征，进程有着创建、活动、暂停、终止等过程，具有生命周期

并发性：多个进程实体同时存在内存中，引入进程的目的就是为了程序与其他程序并发执行

独立性：进程实体是一个能独立运行、独立获得资源和独立接受调度的基本单位

没有建立PCB的程序，都不能作为一个独立单位参与运行

异步性：进程相互制约，进程以不可预知的速度向前推进　所以操作系统中一定要配置响应的进程同步机制

结构性：每个进程都配置一个PCB对其进行描述
- 程序段
- 数据段
- 进程控制段

进程实体

进程的状态与转换

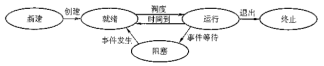
状态

- 运行态：进程在处理机上运行
- 就绪态：进程已处于准备运行状态
- 阻塞态：又称等待态，进程正在等待某个时间而暂停运行
- 创建态：进程正在被创建，尚未进入就绪态
- 结束态：进程正在从系统中消失（包括正常结束或者异常结

相互转换

- 就绪态——>运行态：处于就绪态的进程获得处理机进入运行态
- 运行态——>就绪态：处于运行态的进程时间片用完后，让出处理机进入就绪态
- 运行态——>阻塞态：进程请求除处理机外的其他资源，此时运行态进入阻塞态（系统调用请求操作系统提供服务，这是一种特殊的、由运行用户态程序调用操作系统内核过程的形式）
- 阻塞态——>就绪态：进程等待其他资源的获得，如IO资源、或者中断结束

相互转换



进程控制

进程的创建

- 分配进程标识号，申请PCB（PCB是有限的）
- 为进程分配资源，为程序和数据以及用户栈分配必要的内存空间
- 初始化PCB，包括初始化标志信息、初始化处理机状态信息、初始化处理机控制信息、设置进程的优先级
- 若进程就绪队列可以接纳新进程，进程就进入就绪态

进程的终止

结束分类
- 正常结束：进程的任务已经完成并且准备退出运行
- 异常结束：进程正在运行，出现了某些异常事件，导致程序无法继续运行（存储区越界、保护错、非法指令、特权指令错、IO故障等）
- 外界干预：进程应外界请求终止运行

结束过程
- 根据被终止进程的标识符，检索PCB，读取进程状态
- 若进程处于运行态，终止运行，剥夺处理机
- 终止进程之下的子进程
- 该进程拥有的全部资源还给父进程或者操作系统
- 将PCB从队列中删除

进程的阻塞和唤醒

阻塞原语执行过程
- 找到要被阻塞进程标识号对应的PCB
- 若该进程处于运行态，保护其现场，将其状态转换为阻塞态，停止运行
- 将PCB插入相应时间的等待队列

阻塞是一种自主行为，自我阻塞

唤醒原语的执行过程
- 找到等待队列中进程相应的PCB
- 将其从等待队列中移出，置其状态为就绪态
- 将PCB插入就绪队列，等待调度程序调度

唤醒是被相互有联系的其他进程进行唤醒

进程切换

进程切换是在内核态下完成的

过程
- 保存处理机上下文，包括程序计数器和其他寄存器
- 更新PCB信息
- 把进程的PCB移入相应的队列，如就绪、在某时间阻塞等队列
- 选择另一个进程执行，更新其PCB
- 更新内存管理的数据结构
- 恢复处理机上下文

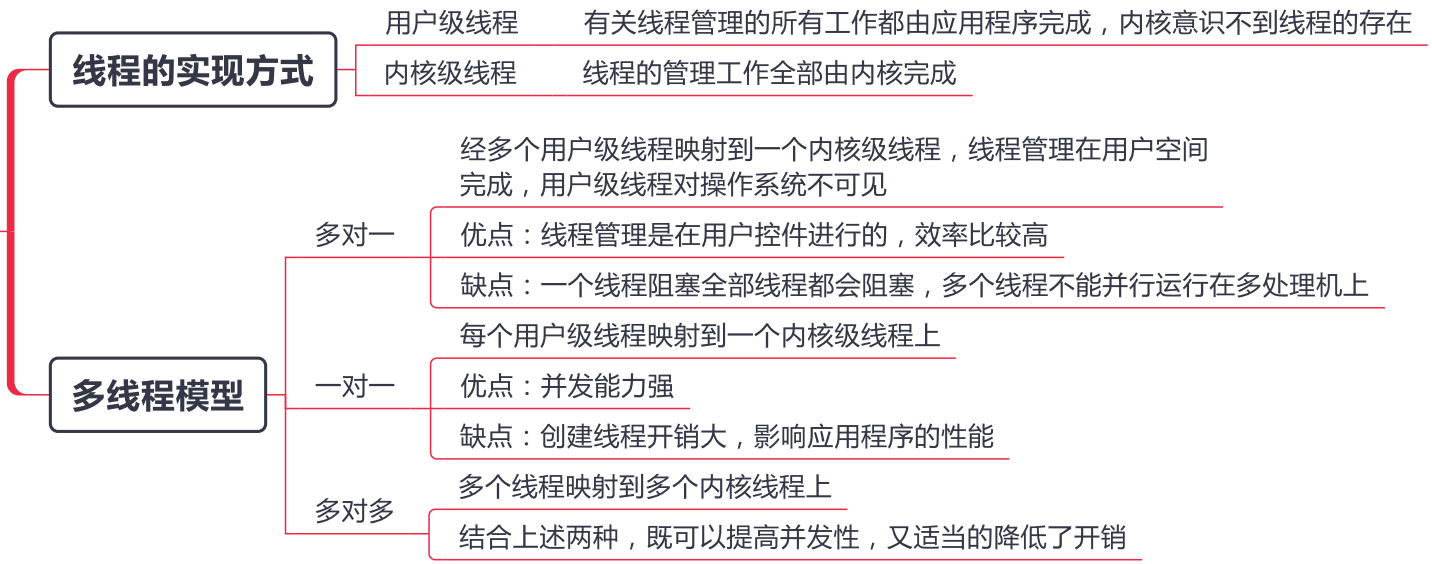
2.1进程与线程（中）

- 进程的组织
  - 进程是一个独立的运行单位，也是操作系统进行资源分配和调度基本单位，由以下三部分构成
  - 进程控制块
    - 进程描述信息
      - 进程标识符：标志进程
      - 用户标识符：进程归属的用户，主要为共享和保护服务
    - 进程控制和管理信息
      - 进程当前状态：描述进程状态信息
      - 进程优先级：描述进程枪战处理机优先级
      - 代码运行入口地址
      - 程序的外存地址
      - 进入内存时间
      - 处理机占用时间
      - 信号量使用
    - 资源分配清单
      - 用以说明有关内存地址空间或者虚拟地址空间状况，所打开的文件的列表和所使用的的输入/输出设备信息
      - 代码段指针、数据段指针、堆栈段指针、文件描述符、键盘、鼠标
    - 处理机相关信息
      - 处理机中各寄存器的值
      - 通用寄存器值、地址寄存器值、控制寄存器值，标志寄存器值，状态字
  - 程序段：能被进程调度程序调度到CPU执行的程序代码段
  - 数据段：进程对应的程序加工处理的原始数据或者程序执行时产生的中间或者最终结果
  - 进程的组织方式
    - 链接方式
      - 按照进程状态将PCB分为多个队列
      - 操作系统持有指向各个队列的指针
    - 索引方式
      - 根据进程状态的不同,建立几张索引表
      - 操作系统持有指向各个索引表的指针
- 进程的通信
  - 共享存储
    - 通信进程之间存在一块可以被直接访问的共享空间
    - 低级方式：基于数据结构共享
    - 高级方式：基于存储区共享

    操作系统只负责为通信进程提供可共享使用的存储空间和同步互斥工具，数据交换则由用户自己安排读/写指令完成
  - 消息传递
    - 进程间的数据交换是以格式化的消息为单位的，进程通过系统提供的发送消息和接收消息的两个原语进行数据交换
    - 直接通信方式：发送进程直接发送消息给接收进程，并将它挂在接收进程的消息缓冲队列上，接收进程从消息缓冲队列中取得消息
    - 间接通信方式：发送进程把消息发送给某个中间实体，接收进程从中间实体中获得消息　例如电子邮件系统
  - 管道通信
    - 发送进程以字符流形式将大量数据送入写管道，接收进程从管道中接收数据
      - 当管道写满时，写进程的write()系统调用将被阻塞，等待读进程将数据取走
      - 当读进程将数据全部取走后，管道变空，此时读进程的read()系统调用将被阻塞
    - 功能：互斥、同步、确定对方存在
    - 半双工通信，不可以同时读和写
    - 限制管道的大小
    - 管道变空的时候阻塞读进程

    管道中的数据被读取后会马上被丢弃
- 线程概念和多线程模型
  - 线程基本概念
    - 减小程序在并发执行时所付出的时空开销，提高操作系统的并发性能
    - 引入线程后，进程只作为系统资源的分配单元，线程作为处理机的分配单元
  - 线程与进程的比较
    - 调度
      - 传统中进程是资源和独立调度的基本单位
      - 引入线程后，进程是独立调度的基本单位，线程是资源的基本单位
      - 不同进程的线程切换会引起进程切换
    - 拥有资源　进程是资源分配的基本单位
    - 并发性　引入线程后，进程可以并发执行，多个线程之间也可以并发执行，提高了系统的吞吐量
    - 系统开销　同一进程的线程切换要比进程切换开销小的多
    - 地址空间和其他资源　进程的地址空间之间相互独立，统一进程的各线程之间共享进程的资源，某进程的线程对其他进程不可见
    - 通信方面
      - 进程间通信需要进程同步和互斥手段的辅助，保证数据的一致性
      - 线程间可以直接读/写进程程序段来进行通信
  - 线程属性
    - 不拥有系统资源，拥有唯一标识符和线程控制块
    - 不同的线程可以执行相同的程序，同一个服务程序被不同用户调用时，操作系统将其创建为不同线程
    - 统一进程的线程共享该进程拥有的全部资源
    - 线程是处理机的独立调度单位
    - 线程也有生命周期，阻塞，就绪，运行等状态
    - 多CPU计算机中,各个线程可占用不同的CPU
    - 每个线程都有一个线程ID、线程控制块（TCB）
    - 切换同进程内的线程,系统开销很小
    - 切换进程,系统开销较大
    - 由于共享内存地址空间,同一进程中的线程间通信甚至无需系统干预

2.1进程与线程（下）


线程的实现方式
用户级线程
有关线程管理的所有工作都由应用程序完成，内核意识不到线程的存在
内核级线程
线程的管理工作全部由内核完成
多线程模型
多对一
经多个用户级线程映射到一个内核级线程，线程管理在用户空间完成，用户级线程对操作系统不可见
优点：线程管理是在用户控件进行的，效率比较高
缺点：一个线程阻塞全部线程都会阻塞，多个线程不能并行运行在多处理机上
一对一
每个用户级线程映射到一个内核级线程上
优点：并发能力强
缺点：创建线程开销大，影响应用程序的性能
多对多
多个线程映射到多个内核线程上
结合上述两种，既可以提高并发性，又适当的降低了开销

2.2处理机调度（上）

- **调度的概念**
  - 调度的基本概念：合理的对进程进行处理机分配
  - 调度的层次
    - 作业调度（高级调度）：从辅存中选择作业送入内存，每个作业只调入一次，调出一次
    - 中级调度（内存调度）：提高内存利用率和系统吞吐量，将暂时不能运行的进程调至外存，使其进入挂起态。或者将已经具备运行条件的进程调入内存，修改其状态为就绪态
    - 进程调度（低级调度）：按照某种策略或者方法从就绪队列中选取一个进程，将处理机分配给它（最基本的调度，频率很高）
  - 三级调度的联系
    - 作业调度为进程活动做准备，进程调度使进程正常活动起来，中级调度将暂时不能运行的进程挂起，中级调度处于作业调度和进程调度之间
    - 作业调度次数少，中级调度次数略多，进程调度频率最高
    - 进程调度使最基本的，不可或缺的
- **调度的时机、切换与过程**
  - 不能切换的情况
    - 处理中断过程
    - 进程在操作系统内核程序临界区的时候
    - 其他需要完全屏蔽中断的原子操作过程
  - 可以切换的情况
    - 发生引起调度条件且当前进程无法继续进行
    - 中断处理结束或者自陷处理结束
- **进程调度方式**
  - 非剥夺调度方式
    - 如果想将处理机分配给一个更高优先级的进程，必须要等待当前占用处理机的进程释放处理机后才能将处理机分配给更高优先级进程
    - 实现简单 开销小 适合大多数批处理系统
    - 不适用于分时系统，和大多数实时系统
  - 剥夺调度方式
    - 如果有更高级进程请求处理机，暂停正在执行的进程，将处理机分配给更高级进程
    - 提高系统吞吐率和响应效率
- **调度的基本准则**
  - CPU利用率，尽可能保持CPU处于忙碌状态
  - 系统吞吐量：单位时间内CPU完成作业的数量，调度算法和方式会对吞吐量造成较大影响
  - 周转时间：作业提交到作业完成的时间
    - 周转时间= 作业完成时间 - 作业提交时间
    - 平均周转时间 = 总周转时间/N个作业
    - 带权周转时间 = 作业周转时间/作业实际运行时间
    - 平均带权周转时间 = 总带权周转时间/N个作业
  - 等待时间：作业等待处理机的时间，衡量一个算法优劣，只需要简单的考察等待时间
  - 响应时间：从用户提交请求到系统首次产生响应所用的时间
- **进程的挂起态与七状态模型**
  - 暂时调到外存等待的进程状态为挂起状态（挂起态）
    - 就绪挂起
    - 阻塞挂起
  - “挂起”和“阻塞”的区别　　两种状态都是暂时不能获得CPU的服务，但挂起态是将进程映像调到外存去了，而阻塞态下进程映像还在内存中

2.2处理机调度（下）

典型调度算法

先来先服务算法
- 作业调度 进程调度
- 先来的先分配处理机
- 优点：算法简单 对长作业有利 有利于CPU繁忙型作业（计算型）
- 缺点：效率低 不利于短作业 不利于IO繁忙型作业
- 不会导致饥饿
- 非抢占式的算法

短作业优先算法
- 进程调度
- 优先选择预计运行时间最短的进程
- 优点：平均等待时间 平均周转时间最短
- 缺点：对长作业不利，造成饥饿现象，没有考虑作业的紧迫性，用户可能可以缩短作业预估时间，使得无法做到短作业优先
- 产生“饥饿”现象。如果一直得不到服务，则称为“饿死”
- SJF和SPF（短进程优先（SPF）算法）是非抢占式的算法。但是也有抢占式的版本——最短剩余时间优先算法

优先级调度算法
- 作业调度 进程调度
- 分类
  - 剥夺型：立即停止当前运行进程，将处理机分配给更高优先级进程
  - 非剥夺型：等待当前进程运行完成，然后将处理机分配给更高优先级进程
- 优先级分类
  - 静态优先级：进程创建后无法对优先级进行修改
  - 动态优先级：可以根据进程运行状态，对进程优先级进行动态调整
- 优先级设置原则
  - 系统进程>用户进程
  - 交互型进程>非交互型进程
  - I/O进程>计算型进程（cpu繁忙型）
- 产生“饥饿”现象
- 有抢占式的，也有非 抢占式的

高响应比调度算法
- 响应比=（等待时间+要求服务时间）/要求服务时间 = 1+等待时间/要求服务时间
- 等待时间相同情况下，要求服务时间越短响应比越大，有利于短作业进程
- 要求服务时间相同，作业响应比由其等待时间决定，等待时间越长响应比越高，实现先来先服务
- 对于长作业，作业的响应比可以随等待时间的增加而提高，等待时间足够长时，其响应比可以升到很高，从而获得处理机
- 不会导致饥饿
- 非抢占式的算法

时间片轮转算法
- 使用与分时系统，使用时间片，就绪进程按照到达先后排成队列，依次在时间片内占用处理机，时间片到达时就释放处理机
- 时间片选择很重要，过大就变成了先来先服务，过短又变成了短作业优先
- 时间片影响因素：系统响应时间，就绪队列中的进程数目和系统的处理能力
- 不会导致饥饿
- 抢占式

多级反馈队列调度算法（融合前面几种算法）
- 实现思想
  - 设置多个就绪队列，为每个队列设置不同的优先级，优先级依次递减
  - 每个队列中的时间片各不相同，时间片依次递增
  - 每个队列按照先来先服务原则进行进程排队，若规定时间片内没有完成，就将进程放入下一级队列
  - 只有到高级队列为空的时候，低等级队列才能开始调度。
- 优点
  - 终端型作业用户：短作业优先
  - 短批处理作业用户：周转时间较短
  - 长批处理作业用户：讲过前面几个队列得到部分执行，不会长期得不到处理
- 产生“饥饿”现象
- 抢占式

2.3进程同步

基本概念

- 临界资源：一次只允许一个进程使用的资源（打印机，特殊变量，数据）
  - 临界资源的访问过程
    - 进入区：检查进程是否可以进入临界区
    - 临界区：可以访问临界资源的代码
    - 退出区：将正在访问临界区的标志清除
    - 剩余区：代码中的其余部分
- 同步：直接制约关系，为了完成某种过任务而建立的多个进程，相互合作，所以要相互进行通信同步
  - 遵循的原则
    - 空闲让进：临界区空闲时，可以允许一个请求进入临界区的进程立即进入临界区
    - 忙则等待：已有进程进入临界区后，其他试图进入临界区的进程必须等待
    - 有限等待：对于请求访问临界区的进程，在有限时间内进入临界区
    - 让权等待：进程不能进入临界区的时候，应当立即释放处理机
- 互斥：间接制约关系，当一个进程访问临界资源的时候，其他进程不能访问

实现临界区互斥的基本方法

- 软件实现方法
  - 单标志法
    - 两个程序进程交替进入临界区
    - 优点：实现简单
    - 缺点：可能会违背空闲让进，造成资源无法充分利用
  - 双标志法先检查
    - 每个进程访问临界区资源前，先检查临界资源是否被访问，如果空闲才能进入
    - 优点：不用交替进入可以连续使用
    - 缺点：两个进程可能同时进入临界区，违背忙则等待
  - 双标志法后检查
    - 先设置自己标志，表明自己想要进入，检查对方标志，如果对方也要进入，那么就等待否则就进入
    - 优点：不会导致两个进程都进入临界区
    - 缺点：双方可能会互相谦让，导致饥饿现象
  - 皮特森算法
    - 防止两个进程无限期等待，在算法三的基础上增加一个标志位，从而防止饥饿
    - 优点：解决了饥饿现象
    - 缺点：算法复杂
- 硬件实现方法
  - 中断屏蔽法
    - 对中断进行屏蔽、关中断
    - 优点：关中断非常方便
    - 缺点：限制了处理机交替执行程序的能力
  - 硬件指令法
    - 读出指定标志后，将该标志置为真

优缺点

- 优点
  - 适用于任意数目的进程
  - 简单且容易验证正确性
  - 支持进程内有多个临界区
- 缺点
  - 不能实现让权等待
  - 可能会导致饥饿现象

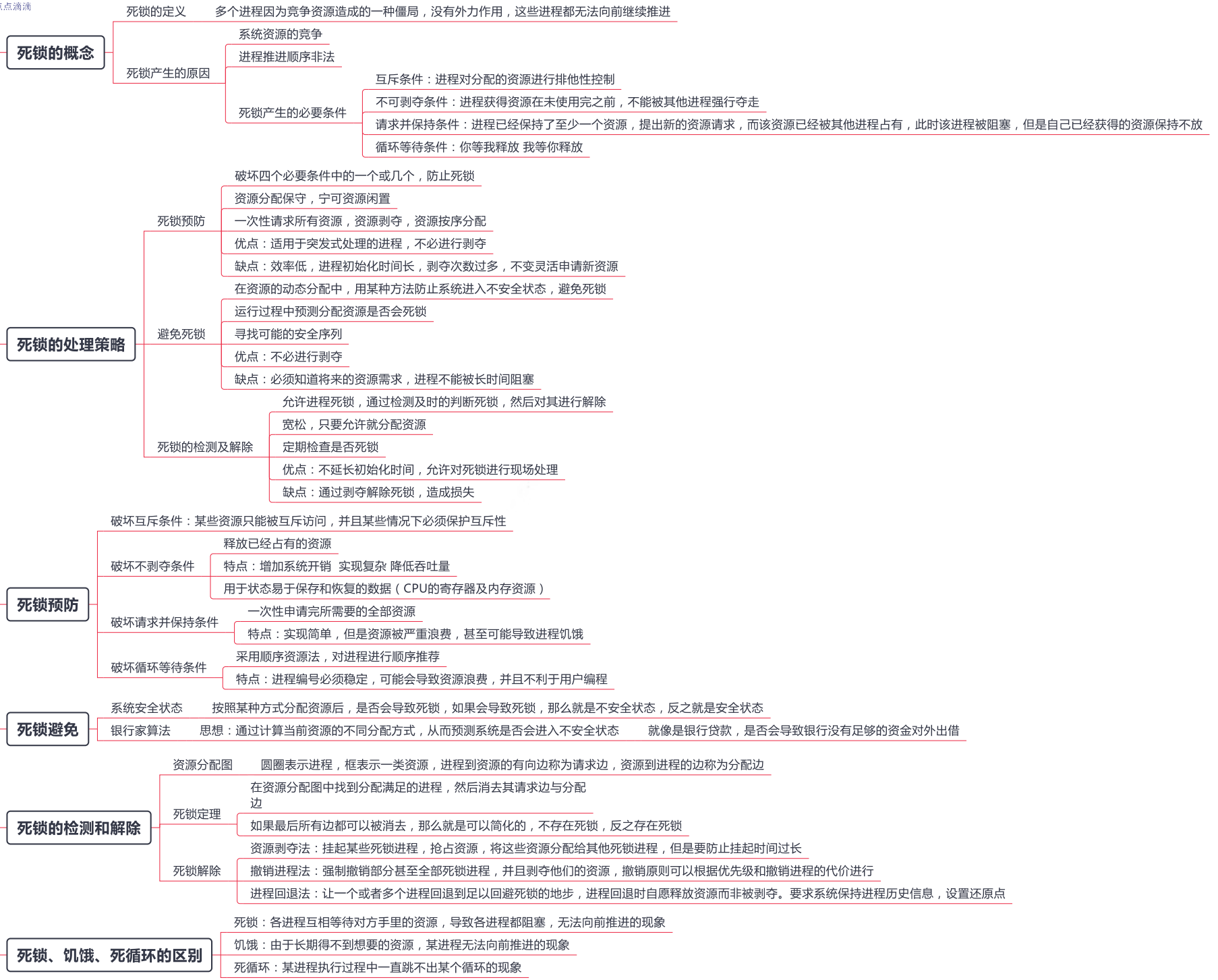
信号量

- 整形信号量
  - wait: 资源-1 signal=资源+1
  - 没有遵循让权等待机制，会导致进程处于“忙等”状态
- 记录型信号量
  - 记录型信号量不存在“忙等”现象，除了需要一个用于代表资源树木的整型变量value外，在增加一个进程链表L，用于链接所有等待该资源的进程
- 利用信号量实现同步
  - 设S为进程P1和P2同步的公共信号量，初值为0，通过设置S的值可以使得P1与P2按照一定顺序执行
- 利用信号量实现互斥
  - 通过设置S的值，可以实现进程对临界资源的互斥访问
- 利用信号量实现前驱关系
  - 通过设置不同的进程运行结束后，产生不同的信号量，从而可以使得目标进程运行，从而实现前驱关系

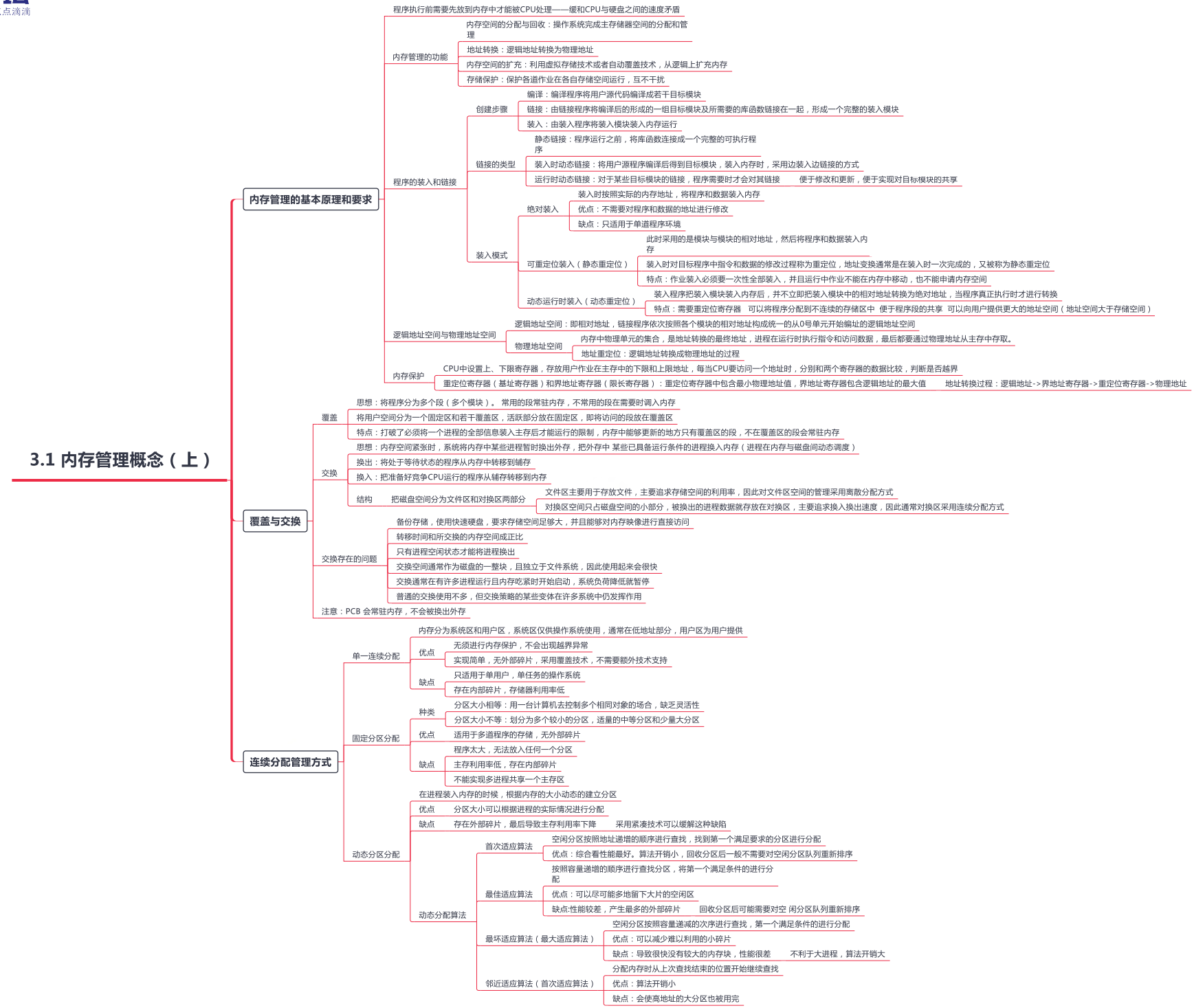
管程

- 定义
  - 一组数据以及定义在这组数据之上的对这组数据的操作组成的软件模块，这组操作能初始化并改变管程中的数据和同步进程
- 组成
  - 局部于管程的共享结构数据说明
  - 对该数据结构进行操作的一组过程
  - 对局部于管程的共享数据设置初始值的语句
- 基本特性
  - 局部于管程的数据只能被局部于管程内的过程所访问
  - 一个进程只有通过调用管程内的过程才能进入管程访问共享数据
  - 每次仅允许一个进程在管程内执行某个内部过程

2.4 死锁


死锁的概念
死锁的定义
多个进程因为竞争资源造成的一种僵局，没有外力作用，这些进程都无法向前继续推进
死锁产生的原因
系统资源的竞争
进程推进顺序非法
死锁产生的必要条件
互斥条件：进程对分配的资源进行排他性控制
不可剥夺条件：进程获得资源在未使用完之前，不能被其他进程强行夺走
请求并保持条件：进程已经保持了至少一个资源，提出新的资源请求，而该资源已经被其他进程占有，此时该进程被阻塞，但是自己已经获得的资源保持不放
循环等待条件：你等我释放 我等你释放
死锁的处理策略
死锁预防
破坏四个必要条件中的一个或几个，防止死锁
资源分配保守，宁可资源闲置
一次性请求所有资源，资源剥夺，资源按序分配
优点：适用于突发式处理的进程，不必进行剥夺
缺点：效率低，进程初始化时间长，剥夺次数过多，不变灵活申请新资源
避免死锁
在资源的动态分配中，用某种方法防止系统进入不安全状态，避免死锁
运行过程中预测分配资源是否会死锁
寻找可能的安全序列
优点：不必进行剥夺
缺点：必须知道将来的资源需求，进程不能被长时间阻塞
死锁的检测及解除
允许进程死锁，通过检测及时的判断死锁，然后对其进行解除
宽松，只要允许就分配资源
定期检查是否死锁
优点：不延长初始化时间，允许对死锁进行现场处理
缺点：通过剥夺解除死锁，造成损失
死锁预防
破坏互斥条件：某些资源只能被互斥访问，并且某些情况下必须保护互斥性
破坏不剥夺条件
释放已经占有的资源
特点：增加系统开销 实现复杂 降低吞吐量
用于状态易于保存和恢复的数据（CPU的寄存器及内存资源）
破坏请求并保持条件
一次性申请完所需要的全部资源
特点：实现简单，但是资源被严重浪费，甚至可能导致进程饥饿
破坏循环等待条件
采用顺序资源法，对进程进行顺序推荐
特点：进程编号必须稳定，可能会导致资源浪费，并且不利于用户编程
死锁避免
系统安全状态
按照某种方式分配资源后，是否会导致死锁，如果会导致死锁，那么就是不安全状态，反之就是安全状态
银行家算法
思想：通过计算当前资源的不同分配方式，从而预测系统是否会进入不安全状态
就像是银行贷款，是否会导致银行没有足够的资金对外出借
死锁的检测和解除
资源分配图
圆圈表示进程，框表示一类资源，进程到资源的有向边称为请求边，资源到进程的边称为分配边
死锁定理
在资源分配图中找到分配满足的进程，然后消去其请求边与分配边
如果最后所有边都可以被消去，那么就是可以简化的，不存在死锁，反之存在死锁
死锁解除
资源剥夺法：挂起某些死锁进程，抢占资源，将这些资源分配给其他死锁进程，但是要防止挂起时间过长
撤销进程法：强制撤销部分甚至全部死锁进程，并且剥夺他们的资源，撤销原则可以根据优先级和撤销进程的代价进行
进程回退法：让一个或者多个进程回退到足以回避死锁的地步，进程回退时自愿释放资源而非被剥夺。要求系统保持进程历史信息，设置还原点
死锁、饥饿、死循环的区别
死锁：各进程互相等待对方手里的资源，导致各进程都阻塞，无法向前推进的现象
饥饿：由于长期得不到想要的资源，某进程无法向前推进的现象
死循环：某进程执行过程中一直跳不出某个循环的现象

3.1 内存管理概念（上）


内存管理的基本原理和要求
程序执行前需要先放到内存中才能被CPU处理——缓和CPU与硬盘之间的速度矛盾
内存管理的功能
内存空间的分配与回收：操作系统完成主存储器空间的分配和管理
地址转换：逻辑地址转换为物理地址
内存空间的扩充：利用虚拟存储技术或者自动覆盖技术，从逻辑上扩充内存
存储保护：保护各道作业在各自存储空间运行，互不干扰
程序的装入和链接
创建步骤
编译：编译程序将用户源代码编译成若干目标模块
链接：由链接程序将编译后的形成的一组目标模块及所需要的库函数链接在一起，形成一个完整的装入模块
装入：由装入程序将装入模块装入内存运行
链接的类型
静态链接：程序运行之前，将库函数连接成一个完整的可执行程序
装入时动态链接：将用户源程序编译后得到目标模块，装入内存时，采用边装入边链接的方式
运行时动态链接：对于某些目标模块的链接，程序需要时才会对其链接　便于修改和更新，便于实现对目标模块的共享
装入模式
绝对装入
装入时按照实际的内存地址，将程序和数据装入内存
优点：不需要对程序和数据的地址进行修改
缺点：只适用于单道程序环境
可重定位装入（静态重定位）
此时采用的是模块与模块的相对地址，然后将程序和数据装入内存
装入时对目标程序中指令和数据的修改过程称为重定位，地址变换通常是在装入时一次完成的，又被称为静态重定位
特点：作业装入必须要一次性全部装入，并且运行中作业不能在内存中移动，也不能申请内存空间
动态运行时装入（动态重定位）
装入程序把装入模块装入内存后，并不立即把装入模块中的相对地址转换为绝对地址，当程序真正执行时才进行转换
特点：需要重定位寄存器　可以将程序分配到不连续的存储区中　便于程序段的共享　可以向用户提供更大的地址空间（地址空间大于存储空间）
逻辑地址空间与物理地址空间
逻辑地址空间：即相对地址，链接程序依次按照各个模块的相对地址构成统一的从0号单元开始编址的逻辑地址空间
物理地址空间
内存中物理单元的集合，是地址转换的最终地址，进程在运行时执行指令和访问数据，最后都要通过物理地址从主存中存取。
地址重定位：逻辑地址转换成物理地址的过程
内存保护
CPU中设置上、下限寄存器，存放用户作业在主存中的下限和上限地址，每当CPU要访问一个地址时，分别和两个寄存器的数据比较，判断是否越界
重定位寄存器（基址寄存器）和界地址寄存器（限长寄存器）：重定位寄存器中包含最小物理地址值，界地址寄存器包含逻辑地址的最大值　地址转换过程：逻辑地址->界地址寄存器->重定位寄存器->物理地址
覆盖与交换
覆盖
思想：将程序分为多个段（多个模块）。 常用的段常驻内存，不常用的段在需要时调入内存
将用户空间分为一个固定区和若干覆盖区，活跃部分放在固定区，即将访问的段放在覆盖区
特点：打破了必须将一个进程的全部信息装入主存后才能运行的限制，内存中能够更新的地方只有覆盖区的段，不在覆盖区的段会常驻内存
交换
思想：内存空间紧张时，系统将内存中某些进程暂时换出外存，把外存中 某些已具备运行条件的进程换入内存（进程在内存与磁盘间动态调度）
换出：将处于等待状态的程序从内存中转移到辅存
换入：把准备好竞争CPU运行的程序从辅存转移到内存
结构　把磁盘空间分为文件区和对换区两部分
文件区主要用于存放文件，主要追求存储空间的利用率，因此对文件区空间的管理采用离散分配方式
对换区空间只占磁盘空间的小部分，被换出的进程数据就存放在对换区，主要追求换入换出速度，因此通常对换区采用连续分配方式
交换存在的问题
备份存储，使用快速硬盘，要求存储空间足够大，并且能够对内存映像进行直接访问
转移时间和所交换的内存空间成正比
只有进程空闲状态才能将进程换出
交换空间通常作为磁盘的一整块，且独立于文件系统，因此使用起来会很快
交换通常在有许多进程运行且内存吃紧时开始启动，系统负荷降低就暂停
普通的交换使用不多，但交换策略的某些变体在许多系统中仍发挥作用
注意：PCB 会常驻内存，不会被换出外存
连续分配管理方式
单一连续分配
内存分为系统区和用户区，系统区仅供操作系统使用，通常在低地址部分，用户区为用户提供
优点
无须进行内存保护，不会出现越界异常
实现简单，无外部碎片，采用覆盖技术，不需要额外技术支持
缺点
只适用于单用户，单任务的操作系统
存在内部碎片，存储器利用率低
固定分区分配
种类
分区大小相等：用一台计算机去控制多个相同对象的场合，缺乏灵活性
分区大小不等：划分为多个较小的分区，适量的中等分区和少量大分区
优点
适用于多道程序的存储，无外部碎片
缺点
程序太大，无法放入任何一个分区
主存利用率低，存在内部碎片
不能实现多进程共享一个主存区
动态分区分配
在进程装入内存的时候，根据内存的大小动态的建立分区
优点　分区大小可以根据进程的实际情况进行分配
缺点　存在外部碎片，最后导致主存利用率下降　采用紧凑技术可以缓解这种缺陷
动态分配算法
首次适应算法
空闲分区按照地址递增的顺序进行查找，找到第一个满足要求的分区进行分配
优点：综合看性能最好。算法开销小，回收分区后一般不需要对空闲分区队列重新排序
最佳适应算法
按照容量递增的顺序进行查找分区，将第一个满足条件的进行分配
优点：可以尽可能多地留下大片的空闲区
缺点:性能较差，产生最多的外部碎片　回收分区后可能需要对空 闲分区队列重新排序
最坏适应算法（最大适应算法）
空闲分区按照容量递减的次序进行查找，第一个满足条件的进行分配
优点：可以减少难以利用的小碎片
缺点：导致很快没有较大的内存块，性能很差　不利于大进程，算法开销大
邻近适应算法（首次适应算法）
分配内存时从上次查找结束的位置开始继续查找
优点：算法开销小
缺点：会使高地址的大分区也被用完

3.1 内存管理概念（中）

非连续分配管理方式

允许一个程序分散的装入不相邻的内存分区

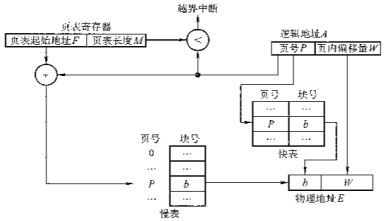
基本分页存储管理方式

设计思想

- 将主存空间划分为大小相等且固定的块，块相对较小，作为主存的基本单位，进程以块为单位尽心空间申请
- 分页存储与固定分区技术很像，但是其分页相对于分区又很小，分页管理不会产生外部碎片，产生的内部碎片也非常的小

分页存储的基本概念

- 页面和页面大小
  - 进程中的块 = 页
  - 内存中的块 = 页框（页帧）
  - 进程申请主存空间，为每个页面分配主存中可用页框，即页与页框一 一对应
  - ***页面大小要适中***
    - 页面太小：进程页面数过多，页表过程，增加内存占用，降低硬件地址转换效率
    - 页面太大：页内碎片过多，降低内存利用率
- 地址结构　　页号（有多少页的编号）+页内偏移（页内存了多少东西）
- 页表
  - 为了便于在内存中找到进程的每个页面对应的物理块，系统为每个进程建立一张页表，记录页面在内存中对应的物理块号，页表一般放在内存中
  - 页表项：页号+物理内存中的块号（不要与地址结构搞混）　　页表项的物理内存块号+地址结构中的页内偏移 = 物理地址

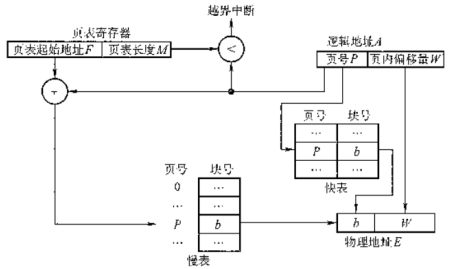


基本地址变换机构

- 计算方式
  - 页号P=A/L，页内偏移量W=A%L
  - 比较页号P和页表长度M，若P>=M产生越界中断
  - 页表中页号P对应的页表项地址=页表始址F+页号P*页表项长度　取出该页表项内容b
  - 计算E=b*L+W　　使用E去访问内存
- 页表项大小的设计应当尽量一页正好能装下所有的页表项
- 分页管理存在的问题
  - 地址变换过程必须足够快，否则访存速率会降低
  - 页表不能太大，否则会降低内存利用率
- 组成
  - 设置一个页表寄存器（PTR），存放页表在内存中的起始地址F 和页表长度M
  - 页表的始址和页表长度放在进程控制块（PCB）中

具有快表的地址变换机构

- 可优化方向：如果页表放在内存中，取地址访问一次内存，按照地址取出数据访问一次内存，共需要两次访问内存
- 优化：地址变换机构中增加一个具有并性查找能力的高速缓冲寄存器（快表），又称为相联存储器（TLB）　　相联存储器既可以按照地址查找也可以按照内容查找
- 访问一个逻辑地址的访存次数
  - 基本地址变换机构　　两次访存
  - 具有快表的地址变换机构
    - 快表命中，只需一次访存
    - 快表未命中，需要两次访存
- 变换过程
  - CPU给出逻辑地址后，先查询快表中是否命中
  - 若快表命中，直接从快表中该页对应的页框好，与页内偏移量拼接成物理地址
  - 若快表不命中，再按照正常方式从页表中查询相应页表项，并将该页表项存入快表中（按照一定策略）
  - ***查找图示***



两级页表

- 如果页数过多，就会导致页表也过多，那么我们可以考虑设置一个用来储存页表的页表(套娃）
- 逻辑地址空间格式 = 一级页号 + 二级页号 + 页内偏移
- 设计多级页表的时候，最后一定要保证顶级页表一定只有一个
- 建立多级页表的目的在于建立索引，不必浪费主存空间去储存无用的页表项，也不用盲目式的查询页表项

3.1 内存管理概念（下）

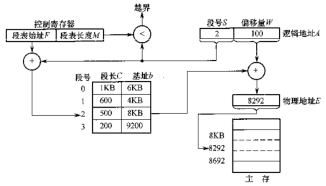
基本分段存储管理方式

出发点
- 分页是从计算机角度考虑设计的，目的是为了内存的利用率，提高计算机性能，分页通过硬件机制实现，对用户完全透明
- 分段是从用户和程序员的角度提出，满足方便编程，信息保护和共享，动态增长及动态链接等多方面的需要

分段
- 按照用户进程中的自然段划分逻辑空间
- 地址结构 = 段号S+段内偏移量W
- 页式系统中，页号和页内偏移对用户透明　　段式系统中 段号和段内偏移量必须由用户显示的提供

段表
- 每个进程都有一张逻辑空间与内存空间映射的段白，这个段表项对应进程的一段，段表项记录该段在内存中的始址和长度
- 段表内容 = 段号 + 段长 + 本段在主存中的地址

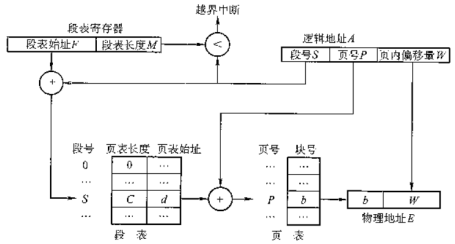
地址变换机构（***变换过程***）
- 逻辑地址A中取出段号S和段内偏移量W
- 比较段号S和段表长度M，若S>=M,则产生越界中断，否则继续执行
- 段号S对应的段表项地址 = 段表始址F+段号S*段表项长度，从该段表项中取出段长C，比较段内偏移量与C的大小判断是否出现越界
- 取出段表项中该段的始址b,计算E = b+W，用得到的物理地址E去访问内存



段的共享与保护
- 共享：两个作业的段表中响应表项指向被共享段的同一个物理副本来实现的　　纯代码或者可重入代码以及不可修改的数据可以被共享
- 保护机制
  - 存取控制保护
  - 地址越界保护

段页式管理方式

- 页式存储有效的提高内存利用率，分段存储能反映程序的逻辑结构并有利于段的分享，将这两种方式结合一下　　这种二者结合的方法经常在计算机理论中遇到
- 思想
  - 作业的地址空间首先被分成若干逻辑段，每段有自己的段号
  - 每个段分成若干大小固定的页
  - 对内存空间的管理仍然和分页存储管理一样
- 地址结构　　段号S+页号P+页内偏移量W
- 为了实现地址变换，系统为每个进程建立了一张段表，每个分段有一个页表　　一个进程中，段表只能有一个，页表可以有多个



- 地址变换方式

补充

- 不能被修改的代码称为纯代码或可重入代码（不属于临界资源）
- 分段与分页的区别
  - 分页对用户不可见，分段对用户可见
  - 分页的地址空间是一维的，分段的地址空间是二维的
  - 分页（单级页表）、分段访问一个逻辑地址都需要两次访存，分段存储中也可以引入快表机构
  - 分段更容易实现信息的共享和保护（纯代码问重入代码可以共享）
- 分段与分页优缺点
  - 分页管理
    - 优点：内存空间利用率高，不会产生外部碎片，只会有少量的页内碎片
    - 缺点：不方便按照逻辑模块实现信息的共享和保护
  - 分段管理
    - 优点　　很方便按照逻辑模块实现信息的共享和保护
    - 缺点
      - 如果段长过大，为其分配很大的连续空间会很不方便
      - 段式管理会产生外部碎片

3.2虚拟内存管理（上）


虚拟内存的基本概念
传统存储管理方式的特征
一次性：作业必须一次性全部装入内存后，才能开始运行
作业很大无法装入则无法运行
大量作业要求运行时，由于内存不足，只能一部分作业先运行，导致多道程序度下降
驻留性：作业装入内存后，一直驻留在内存中，任何部分不会被换出。
局部性原理
时间局部性
一条指令执行后，不就之后指令可能被再次执行，数据被访问后，不久后数据可能再次被访问
原因：程序中存在着大量的循环操作
时间局部性通过将最近使用的指令和数据存储在高速缓冲存储器中
空间局部性
一旦程序访问了某个存储单元，不久之后附近的存储单元也将被访问
原因：指令通常是顺序存放，顺序执行的，数据一般也是以向量、数组、表等形式簇聚存储的
空间局部性使用较大的高速缓存，将预取机制继承到高速缓存控制逻辑中实现
虚拟存储器的定义和特征
基于局部性原理，程序的一部分装入内存，一部分留在外存，需要的时候将外存内容调入内存，就好像产生了一个巨大的内存空间
特征
多次性：作业在运行时，分多次调入内存运行
对换性：作业不必一直驻留内存，允许作业在运行过程中进行换进换出
虚拟性：从逻辑上扩充内存容量，使用户看到的内存容量远大于实际的内存容量
虚拟内存技术的实现
建立在离散分配的内存管理方式上
实现方式
请求分页存储管理
请求分段存储管理
请求段页式存储管理
硬件支持
一定容量的内存和外存
页表机制（或者段表机制）
中断机构
地址变换机构
请求分页管理方式
系统建立在基本分页系统基础之上，为了支持虚拟存储器功能而增加了请求调页功能和页面置换功能
页表机制
组成：页号 物理块号 状态位P 访问字段A 修改位M 外村地址
状态位：当前页是否已经调入内存
访问字段A：记录本页在一段时间内被访问的次数
修改位M：记录本页是否被修改过
外存地址：指出该页在外存上的位置（通常是物理块号）
缺页中断机构
当访问页面不在内存时就会产生缺页中断
特点
指令执行期间产生中断，而不是指令执行之后产生中断和处理中断
一条指令在执行期间，可能产生多次缺页中断
地址变换机构
检索快表，找到访问页，修改页表项中的访问位，利用页表项中给出的物理块号和页内地址形成物理地址
没有找到改页的页表项，去内存中寻找页表，看该页是否已经调入内存，没有调入则产生缺页中断，请求从外存把该页调入内存
易混知识点
虚拟内存的最大容量是由计算机的地址结构（CPU寻址范围）确定的
虚拟内存的实际容量 = min（内存和外存容量之和，CPU寻址范围）

3.2虚拟内存管理（下）


页面置换算法
最佳置换算法（OPT）
选择永不使用或者最长时间内不再访问的页面进行淘汰，但是现实中是无法预知的
优点：缺页率最小，性能最好
先进先出页面置换算法（FIFO）
优先淘汰最早进入的页面
优点：实现简单
缺点：与进程的实际运行规律不匹配
Belady异常：增大分配的物理块数但是故障数不减反增　只有先进先出算法会出现
最近最久未使用（LRU）置换算法
选择最近最长时间没有被访问的页面进行淘汰，每个页面设置一个访问字段，用来标识上次被访问到现在经历的时间
优点：性能好
缺点：实现复杂 需要寄存器和栈的硬件支持 LRU是堆栈类算法
时钟(CLOCK)置换算法
像一个时钟一样转圈，每个页面设置一个使用位（访问位），遇到没有被使用的就会将页面换出，然后将使用位置0，如果遇到使用的就会将使用位置零，然后扫描下一个
优点：性能接近于最佳置换算法
缺点：实现复杂 开销大
改进型CLOCK算法
使用位（访问位）的基础上增加修改位
扫描过程
扫描缓冲区，选择第一个使用位和修改位都为0的页面换出
第一步失败后，查找使用位为0，修改位为1的进行替换，对于每个跳过的帧，将使用位置为0
第二步失败后，指针回到初始地点且使用位（访问位）均为0，重复第一步
优点：相对于未改进型，节省了时间
页面分配策略
驻留集：给一个进程的分配的物理页框的集合就是这个进程的驻留集
考虑因素
分配给一个进程的的存储量越小，任何时候驻留在主存中的进程数就越多，可以提高处理机的时间利用率
一个进程在主存中的页数过少，页错误率就会相对较高
页数过多，对进程的错误率也不会产生过多的影响
分配策略
固定分配局部置换
每个进程分配固定物理块数，缺页的时候就进行换页
难以确定每个进程应该分配的物理块数
太多导致资源利用率下降 太少导致频繁缺页中断
可变分配全局置换
进程分配一定物理块，系统自身保留一定空闲物理块，如果进程缺页，就对该进程分配新的物理块
优点：最容易实现，动态调整物理块分配
缺点：如果盲目分配物理块，就会导致多道程序并发能力下降
可变分配局部置换
根据进程的缺页情况，对物理块进行动态分配，如果频繁缺页，就对其多分配物理块，如果缺页率特别低，就减少其物理块
优点：保持了系统的多道程序并发能力
缺点：增大了开销，实现复杂
调入页面的时机
预调页策略
将预计不久被访问的页面调入，成功率约为50%
请求调页策略
当进程提出缺页的时候，再按照一定策略进行调页
特点：一次调入一页，调入/调出页面数多时会花费过多的I/O开销
一般情况下两种策略同时使用
从何处调入页
拥有足够的对换空间　可以全部从对换区调入所需页面，提高调页速度
缺少足够的对换区空间　不会被修改的文件从文件区调入，可能被修改的部分换入对换区，以后再从对换区调入　原理：读速度比写速度块
UNIX方式　进程相关文件访问文件区，没有运行的页面从文件区调入，曾经运行过但又被换出的页面放在对换区
抖动
刚换出的页面又要换入内存
原因
分配的物理页帧数不足（主要原因）
置换算法不当
工作集
某段时间内，进程要访问的页面集合。
原理
操作系统跟走每个进程的工作集，并为进程分配大于其工作集的物理块
落入工作集的页面需要调入驻留集中，落在工作集外面的页面可以从驻留集中换出
若还有空闲物理块，可以再调入一个进程到内存以增加多道程序数。
若所有进程的工作集之和超过了可用物理块的总数，操作系统就会暂停一个进程，并将其页面调出并将其物理块分配给其他进程

4.1文件系统基础（上）

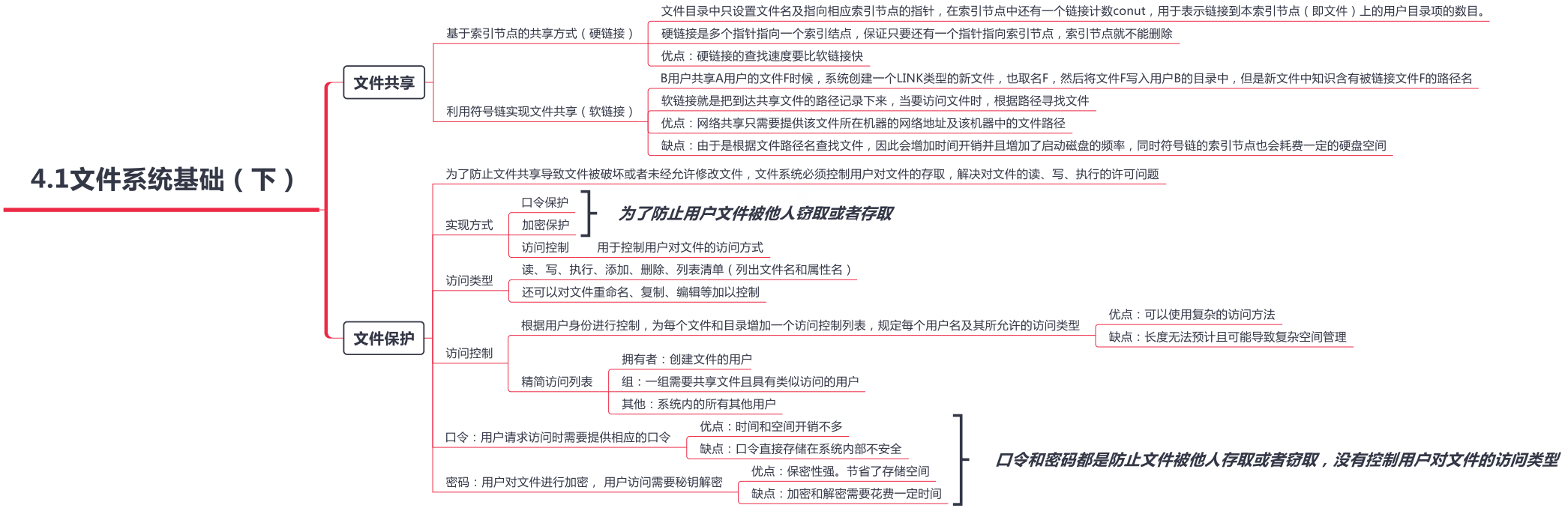
- **文件的相关概念**
  - 相关定义
    - 文件是以计算机硬盘为载体的存储在计算机上的信息集合，文件可以是文本文档、图片、程序等
    - 系统运行时，计算机以进程为基本单位进行资源的调度和分配
    - 在用户输入输出时，以文件为基本单位
    - 操作系统的文件系统：用于实现文件的权限访问，修改，查询和保存等功能
    - 文件的结构
      - 数据项：数据项是文件系统中最低级的数据组织形式
        - 基本数据项：用于描述一个对象的某种属性的一个值
        - 组合数据项：多个基本数据项组成
      - 记录：一组数据项的集合，用于描述一个对象在某方面的属性
      - 文件：创建者所定义的一组相关信息的集合
        - 有结构文件：相似的记录组成（记录式文件）
        - 无结构式文件：字符流组成（流式文件）
  - 文件的属性
    - 名称：文件名称唯一，以容易读取的形式保存
    - 标识符：文件的唯一标签，通常为数字，是对人不可读的一种内部名称
    - 类型：被支持的不同类型的文件系统所使用
    - 位置：指向设备和设备上文件的指针
    - 大小：文件当前的大小，包含文件允许的最大值
    - 保护：对文件进行保护的访问控制信息
    - 时间、日期和用户标识：文件创建、修改和上次访问的相关信息，用于保护和跟踪文件的使用
  - 文件的基本操作
    - 创建文件
      - 文件系统为文件找到空间
      - 目录中为文件创建条目，该条目记录文件名称、在文件系统中的位置以及其他可能的信息
    - 写文件：执行系统调用，指明文件名称和写入内容，查找文件位置，为该文件维护一个写位置的指针，当发生写操作的时候更新写指针
    - 读文件：执行系统调用，指出文件名称和文件位置，搜索目录项，系统维护一个读指针，发生读操作就对该指针进行更新
    - 文件重定位（文件寻址）：按照某种条件搜索目录，将当前文件位置设为定值。并且不会读、写文件
    - 删除文件：搜索目录，找到文件的目录项，使其变为空项，然后回收目标文件占用的存储空间
    - 截断文件：允许文件的所有属性不变，并删除文件内容，即将其长度设为0并释放其空间
  - 文件的打开与关闭
    - open请求
      - 首次使用文件，会调用open请求指明文件的属性（包括其物理位置）从外存复制到内存打开文件表的一个表目中，并将该表目的编号（索引）返回给用户
      - 操作open会根据文件名搜索目录，并将目录条目复制到打开文件
        - 调用open请求（创建、只读、读写、添加等）得到允许，进程就可以打开文件，open会返回一个指向打开文件表中的一个条目的指针
        - 通过使用该指针进行I/O操作，简化步骤并节省资源
    - 文件关联信息
      - 文件指针：系统跟踪上次的读写位置作为当前文件位置的指针，这种指针对于打开文件的某个进程来说是唯一的，因此必须与磁盘文件属性分开保存
      - 文件打开计数：文件关闭时，必须重用其打开文件表条目，否则表内空间会不够用，计数器为0关闭文件，删除该条目
      - 文件磁盘位置：该信息存储在内存放，以免每个操作都要从磁盘中读取
      - 访问权限：每个进程打开文件都需要一个访问模式（创建、只读、读写、添加等）。该信息保存在进程打开的文件表中，以便操作系统能够允许或拒绝之后的I/O请求
- **文件的逻辑结构**
  - 无结构文件
    - 最简单的文件组织形式
    - 将数据按照顺序组织记录并积累、保存、是有序相关信息项的集合
    - 由于其没有结构，所以只能采用更穷举搜索
    - 管理简单，方便用户对其操作
    - 基本信息单位操作不多的文件适合采用字符流的无结构方式
  - 有结构文件（记录式文件）
    - 顺序文件
      - 文件的记录是一个接一个排列，记录通常是定长的，可以顺序存储或者链表存储
      - 批量处理时，顺序文件的效率是所有逻辑文件中效率最高的
      - 但是增删改查操作比较困难
    - 索引文件
      - 定长记录文件：按照公式A = i*L可以直接得到文件地址（第i条记录，L是文件长度）
      - 变长记录文件
        - 查找前i-1条记录后，才能查找第i条记录
        - 通过建立索引表后可以有效提高查找速度
    - 索引顺序文件
      - 顺序和索引两种组织形式的结合。
      - 索引文件将顺序文件中的所有记录分成若干组，为顺序文件建立起一张索引表，在索引表中为每组中的第一条记录建立一个索引项，其中含有该记录得关键字值和指向该记录的指针
      - 索引顺序文件提高了查找效率，但是索引表也占用了存储空间
    - 直接文件或散列文件
      - 给定记录的键值或通过散列函数转换的键值直接决定记录的物理地址
      - 这种映射结构不同于顺序文件或者索引文件，没有顺序的特性

4.1文件系统基础（中）

目录结构

- 包含有关文件的信息，如属性、位置和所有权等
- 文件控制块和索引结点
  - 文件控制块（FCB）
    - 用来存放控制文件需要的各种信息的数据结构，实现“按名存取”。
    - 包含信息
      - 基本信息：文件名，文件的物理位置，逻辑结构、物理结构等
      - 存取控制信息：文件存取权限
      - 使用信息：文件建立时间 修改时间
  - 索引结点
    - 检索目录文件时，不需要将文件调入内存，只是查找其目录项，文件的描述信息单独形成为索引节点的数据结构
    - 磁盘索引节点
      - 文件主标识符：拥有该文件的个人或小组的标识符
      - 文件类型：普通文件、目录文件、特别文件
      - 文件存取权限：各类用户对该文件的存取权限
      - 文件物理地址：每个索引节点中含有13个地址项，直接或者间接的方式给出数据文件所在盘块的编号
      - 文件长度：字节为单位
      - 文件链接计数:本文件系统中所有指向该文件的文件名的指针计数
      - 文件存取时间：文件最近被进程存取，修改以及索引节点最近被修改的时间
    - 文件打开后内存索引节点增加的内容
      - 索引结点编号：用于标识内存索引节点
      - 状态：指示i节点是否被上锁或者被修改
      - 访问计数：每当有一个进程要访问此i结点时，计数加1，访问结束减1
      - 逻辑设备号：文件所属文件系统的逻辑设备号
      - 链接指针：设置分别指向空闲链表和散列队列的指针
- 目录结构
  - 搜索：用户使用给一个文件时，需要搜索目录，找到该文件对应的目录项
  - 创建文件：创建一个新文件时，需要在目录中增加一个目录项
  - 删除文件：删除一个文件时，需要在目录中删除相应的目录项
  - 显示目录：用户可以请求显示目录的内容，显示该用户目录中的所有文件及属性
  - 修改目录：某些文件属性保存在目录中，因而这些属性的变化需要改变相应的目录项
- 目录结构分类
  - 单机目录结构
    - 整个文件系统只建立一张目录表，每个文件占一个目录项
    - 优点：实现了按名存取
    - 缺点：查找速度慢，文件不允许重名，不便于文件共享，不适用于多用户的操作系统
  - 两级目录结构
    - 将文件分为主目录和用户目录，主目录记录用户名及相应用户文件目录所在的存储位置，用户目录项记录该用户文件的FCB信息。
    - 优点：解决了不同用户文件重名问题，在一定程度上保证了文件的安全
    - 缺点：缺乏灵活性，不能对文件分类
  - 多级目录结构
    - 将两级目录结构的层次关系加以推广，就形成了多级目录结构，即树形目录结构
    - 进程对各文件的访问都是相对于当前目录进行的
    - 优点：有效的对文件进行分类，文件结构层次清晰，能够有效的进行文件管理和保护
    - 缺点：按照路径名访问中间结点，增加了磁盘访问次数，降低了查询速度
  - 无环图目录结构
    - 在树形目录结构基础上增加了一些指向同一结点的有向边，使整个目录称为一个有向无环图。
    - 优点：有利于实现文件共享

4.1文件系统基础（下）


文件共享
基于索引节点的共享方式（硬链接）
文件目录中只设置文件名及指向相应索引节点的指针，在索引节点中还有一个链接计数conut，用于表示链接到本索引节点（即文件）上的用户目录项的数目。
硬链接是多个指针指向一个索引结点，保证只要还有一个指针指向索引节点，索引节点就不能删除
优点：硬链接的查找速度要比软链接快
利用符号链实现文件共享（软链接）
B用户共享A用户的文件F时候，系统创建一个LINK类型的新文件，也取名F，然后将文件F写入用户B的目录中，但是新文件中知识含有被链接文件F的路径名
软链接就是把到达共享文件的路径记录下来，当要访问文件时，根据路径寻找文件
优点：网络共享只需要提供该文件所在机器的网络地址及该机器中的文件路径
缺点：由于是根据文件路径名查找文件，因此会增加时间开销并且增加了启动磁盘的频率，同时符号链的索引节点也会耗费一定的硬盘空间
文件保护
为了防止文件共享导致文件被破坏或者未经允许修改文件，文件系统必须控制用户对文件的存取，解决对文件的读、写、执行的许可问题
实现方式
口令保护
加密保护
为了防止用户文件被他人窃取或者存取
访问控制
用于控制用户对文件的访问方式
访问类型
读、写、执行、添加、删除、列表清单（列出文件名和属性名）
还可以对文件重命名、复制、编辑等加以控制
访问控制
根据用户身份进行控制，为每个文件和目录增加一个访问控制列表，规定每个用户名及其所允许的访问类型
优点：可以使用复杂的访问方法
缺点：长度无法预计且可能导致复杂空间管理
精简访问列表
拥有者：创建文件的用户
组：一组需要共享文件且具有类似访问的用户
其他：系统内的所有其他用户
口令：用户请求访问时需要提供相应的口令
优点：时间和空间开销不多
缺点：口令直接存储在系统内部不安全
密码：用户对文件进行加密， 用户访问需要秘钥解密
优点：保密性强。节省了存储空间
缺点：加密和解密需要花费一定时间
口令和密码都是防止文件被他人存取或者窃取，没有控制用户对文件的访问类型

4.2文件系统的实现

- **文件层结构**
  - 用户调用接口：文件系统为用户提供与文件及目录有关的调用
  - 文件目录系统：管理文件目录，管理活跃文件目录表，管理读写状态信息表，管理用户进程的打开文件表，管理与组织存储设备上的文件目录结构。调用下一级存取控制模块
  - 存取控制验证：实现文件保护，将用户的访问请求与FCB中指示的访问控制权限进行比较，以确认访问的合法性
  - 逻辑文件系统关于文件信息缓冲区：逻辑文件系统与文件信息缓冲区的主要功能是，根据文件的逻辑结构将用户要读写的逻辑记录转换成文件逻辑结构内的相应块号
  - 物理文件系统：把逻辑记录所在的相对块号转换成实际的物理地址
  - 辅助分配模块：管理辅存空间，负责分配辅存空闲空间和回收辅存空间
  - 设备管理程序模块：分配设备，分配设备读写用缓冲区，磁盘调度，启动设备，处理设备中断，释放设备读写缓冲区，释放设备
- **目录实现**
  - 线性列表
    - 使用存储文件名和数据块指针的线性表
    - 优点：实现简单
    - 缺点：耗费时间
  - 哈希表
    - 根据文件名得到一个值，然后返回一个指向线性列表中元素的指针
    - 优点：查找迅速，插入和删除简单
    - 缺点：要避免充足，哈希表长度固定以及哈希函数对表长有依赖性
- **文件实现**
  - 文件的分配方式
    - 连续分配
      - 每个文件在磁盘上占有一组连续的块，磁盘地址定义了磁盘上的一个线性排序。
      - 访存1次
      - 优点：实现简单，存取速度快，使得访问磁盘需要的寻道数和寻道时间最小
      - 缺点：文件长度不宜动态的增加，会产生外部碎片
    - 链接分配
      - 采用离散分配方式，提高了磁盘空间利用率　　消除了外部碎片
      - 访存n次
      - 隐式链接
        - 磁盘块分布在磁盘的任何地方，除最后一个盘块，其他盘块都有指向下一个盘块的指针
        - 优点：不会有碎片问题，外存利用率高
        - 缺点：不能直接访问　稳定性存在问题
      - 显示链接
        - 把用于链接文件各物理块的指针，从每个物理块的末尾提取出来，显示的存放在内存的一张连接表中。整个磁盘设置一张
        - 优点：显著的提高检索速度，减少了访问磁盘次数
        - 缺点：文件分配表的需要占用一定的存储空间
    - 索引分配
      - 索引分配解决了链接分配不能直接访问的问题
      - m级要访存m+1次
      - 优化机制
        - 链接方案:一个索引块通常为一个磁盘块，为了处理大文件，可以将多个索引块链接起来
        - 多层索引：第一层索引块指向第二层索引块，第二层索引块，指向文件块
        - 混合索引　　系统既采用直接地址有采用单级索引分配方式或者两级索引分配方式
  - 文件存储空间管理
    - 文件存储在一个文件卷中，文件卷可以是物理盘的一部分，也可以是整个物理盘　　在一个文件卷中，文件数据信息的（文件区）和存放文件控制信息FCB的空间是分离的
    - 文件存储设备分成许多大小相同的物理块，以块为单位交换信息　　文件存储设备管理的实质是对空闲块的组织和管理，包括空闲块的组织、分配与回收等问题
    - 空闲块管理
      - 空闲表法：属于连续分配方式，系统为空闲区建立一张空闲盘块表，每个空闲区第一个盘块号，该区的空闲盘块数等信息。
      - 空闲链表法：将所有的空闲盘区拉成一条空闲链，根据构成链所有的基本元素不同，可以把链表分成两种形式：
        - 空闲盘块链：将磁盘上所有空闲空间以盘块为单位拉成一条链
        - 空闲盘区链：将磁盘上所有空闲盘区拉成一条链
    - 位示图法　采用二进制的一位来表示一个盘块的使用情况，磁盘上所有的盘块都有一个二进制位与之对应
      - 盘块的分配　计算公式：b = n(i-1)+j
      - 盘块的回收
        - i = （b-1) DIV n + 1
        - j = (b-1) MOD n + 1
      - *i行j列*
    - 成组链接法
      - UNIX使用，结合了空闲表和空闲链表法克服了表太大的缺点
      - 把顺序的n个空闲扇区地址保存在第一个空闲扇区内，其后一个空闲扇区内则保存另一顺序空闲扇区的地址

王道论坛
专注于计算机考研的点点滴滴

4.3 磁盘组织与管理

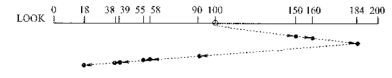
- **磁盘结构**
  - 表面涂有磁性物质的金属或塑料构成的圆形盘片，通过一个称为磁头的导体线圈从磁盘读取数据。
  - 磁盘盘面上的数据存储在一组同心圆中，称为磁道
  - 一个盘面有上千个磁道，磁道又划分为几百个扇区，每个扇区固定存储大小，一份扇区称为一个盘块
  - 磁盘地址用　柱面号-盘面号-扇区号(块号）　表示
  - 磁盘分类
    - 固定头磁盘：磁头相对于盘片的径向方向固定
    - 活动头磁盘：每个磁道一个磁头，磁头可以移动
    - 固定盘磁盘：磁头臂可以来回伸缩定位磁道，磁盘永久固定在磁盘驱动器内
    - 可换盘磁盘：可以移动和替换
- **磁盘调度算法**
  - 读写时间组成
    - 寻找时间：活动头磁盘在读写信息前，将磁头移动到指定磁道所需要的时间（跨越n条磁道+启动磁臂）：Ts = m*n +s
    - 延迟时间：磁头定位到某一磁道扇区所需要的时间，Tr = 1/2r（转速为r）
    - 传输时间：从磁盘读出或向磁盘写入数据经过时间， Tr=b　/ rN
  - 磁盘调度算法
    - 先来先服务算法（FCFS）
      - 按照进程请求访问磁盘的先后顺序进行调度
      - 优点：公平　实现简单
      - 缺点：适用于少量进程访问，如果进程过多算法更倾向于随机调度
    - 最短寻找时间有限算法（SSTF)
      - 选择调度处理的磁道是与当前磁头所在磁道距离最近的磁道
      - 优点：性能强于先来先服务算法
      - 缺点：容易产生饥饿现象
    - 扫描(SCAN)算法
      - 在磁头当前移动方向上选择与当前磁头所在的磁道距离最近的请求作为下一次服务对象
      - 优点：寻道性能好，可以避免饥饿现象
      - 缺点：对最近扫描过的区域不公平，访问局部性方面不如FCFS和SSTF好
    - 循环扫描算法（C-SCAN)　磁头单向移动，回返时直接回到起始端，而不服务任何请求
    - LOOK与C-LOOK算法
      - 在SCAN与C-SCAN算法的基础上规定了查看移动方向上是否有请求，如果没有就不会继续向前移动，而是直接改变方向（LOOK）或者直接回到第一个请求处（C-LOOK)
      - 

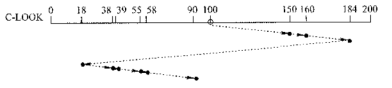
        图 4.23-1　LOOK 磁盘调度算法
      - 

        图 4.23-2　C-LOOK 磁盘调度算法
    - 对盘面交替编号　原因：磁头在读/写一个物理块后，需要经过短暂的处理时间才能开始处理下一块
- **磁盘的管理**
  - 磁盘初始化
    - 低级格式化：磁盘分扇区，为每个扇区采用特别的数据结构（头、数据区域、尾部组成）,头部含有一些磁盘控制器所使用的信息
    - 进一步格式化处理：磁盘分区，对物理分区进行逻辑格式化（创建文件管理系统），包括空闲和已分配的空间及一个初始为空的目录
  - 引导块
    - 计算机启动时运行自举程序，初始化CPU寄存器、设备控制器和内存等，然后启动操作系统
    - 组局程序通常保存在ROM中，在ROM中保留很小的自举块，完整的自举程序保存在启动块上
    - 拥有启动分区的磁盘称为启动磁盘或系统磁盘
  - 坏块
    - 无法使用的扇区
    - 对于简单的磁盘，可以在逻辑格式化时（建立文件系统时）对整个磁盘进行坏块检查，标明哪些扇区是坏扇区，比如：在 FAT 表上标明
    - 处理方式
      - 简单磁盘：手动处理，对坏块进行标记，程序不会使用
      - 复杂磁盘:控制器维护一个磁盘坏块链表，同时将一些块作为备用，用于替代坏块（扇区备用）